

Google Summer of Code - 2023

[Pitivi]



Project: Migrate Pitivi from GTK3 to GTK4

Personal Details

- Name: Jainil Patel
- Time Zone: India (UTC+5.30)
- Telephone: +91 9904892000
- Email: jainilpatel2003@gmail.com
- IRC Nick: Jainil Patel
- GitLab: [jainil28patel](https://gitlab.com/jainil28patel)

University Details

University: Indian Institute of Technology, Roorkee, India
Major: Computer Science and Technology
Current Year: 2nd year (2025 expected graduation)
Degree: Bachelor of Technology (B.Tech)

What is your email address and IRC nick?

Email Address : jainilpatel2003@gmail.com

IRC Nick : Jainil Patel

What is your blog address?

My blog is hosted on GitHub pages at <https://jainl28patel.github.io>

What city and country will you reside in during the summers?

I will be living in [Roorkee, India](#) (University) or [Ahmedabad, India](#) (Home) during the summers depending on my university summer break dates.

What is your academic background?

Below I have mentioned my academic background.

- University : Indian Institute of Technology, Roorkee
- Major : Computer Science and Engineering
- Current Year : Sophomore (2nd Year)
- Expected Graduation : 2025
- Degree : Bachelor of Technology (B.Tech)
- CGPA : 9.01 (out of 10)

Who is the possible mentor for the project you are proposing?

My possible mentor is [Alexandru Bălut \(aleb\)](#), he is a maintainer and knows everything about the UI of Pitivi. As my project aims towards migrating UI from GTK 3.0 to GTK 4.0 he is the possible mentor for my project.

What is the ultimate goal of your proposal?

Pitivi's UI is in GTK, an open-source, cross-platform widget toolkit. Currently, Pitivi uses GTK 3.0 but the current new and stable version of GTK is GTK 4.0, at some point of time GTK3.0 will be deprecated and hence it is necessary to migrate Pitivi to GTK 4.0. The ultimate goal of my project will be to continue migrating Pitivi's codebase to GTK 4.0 runtime and then migrate the app to GTK 4.0.

What applications/libraries will the proposed work modify or create?

My proposed work will not add any new application/library but It will modify Pitivi's existing code-base including the [UI](#) files, main [Python code-base](#) and [Unit Tests](#).

What benefits does your proposed work have for the Pitivi/Gstreamer/GNOME community?

Currently, Pitivi's UI is written in GTK 3, but the GTK 4 released in December 2020 is far superior and provides much more functionalities and better performance compared to GTK 3. GTK 4 provides the following advancements compared to GTK 3.

1. Improved performance: GTK4 has been designed to use a new rendering engine based on the Vulkan graphics API which enables more efficient use of the GPU and results in the faster rendering of complex user interfaces. It has also included new widgets like **GtkGLArea Widget** which uses GL API to compile and use shaders.
2. Better support for touch and gestures: Important as more and more devices are designed with touchscreens.

3. Simplified API: GTK 4 has removed many redundant APIs which were there in GTK 3 which makes the development process easier.
4. Improved Documentation

Due to the above-mentioned advantages and for providing better user experience, more support and new scopes of the possibility of a feature addition and improvements in Pitivi and to stay up to date with the current well-maintained and heavily used tech stack it becomes very important to migrate Pitivi from GTK3 to GTK4 which will benefit both the developers and users for Pitivi providing with better user experience, ease of maintenance and opening possibilities for future developments and feature additions.

Why are you the right person to work on this project?

This project requires experience in writing UI for desktop apps using GTK in Python, and I believe I have the necessary experience to be the right person for the job. I have previously worked on developing desktop apps along with their UI in different frameworks using Python, which has given me a good understanding of how to develop user interfaces that are intuitive and user-friendly.

Moreover, I understand that during the integration and testing phase, a lot of bugs might come up in the UI, and testing and debugging could be done faster and in an effective manner if the developer has prior experience working on such a large code base. I have worked on such large code bases in the past, like in Godot and Rootex (explained [here](#)), which has given me a good understanding of how to navigate and debug complex codebases.

I am a part of [SDSLabs](#) (student run technical group), at IITR focused on software development and CTFs. Working on projects undertaken by SDS Labs, I know how to work in a group while developing a piece of software, the effective use of version control systems, and the importance of communication.

In conclusion, I have the necessary experience in developing user interfaces for desktop applications using Python, as well as working with complex code bases. With my experience

in SDSLabs, I have also gained important teamwork and communication skills. I believe that my skills and experience make me a strong fit for this project.

How do you plan to achieve completion of your project?

Primary Goals

The Project is majorly divided into three subsections namely

1. Migrating code from GTK3 to GTK4
2. Updating the unit test
3. Bug fixing

Migrating code from GTK3 to GTK4

https://gitlab.gnome.org/GNOME/pitivi/-/merge_requests/442 (Merge request)

<https://docs.gtk.org/gtk4/migrating-3to4.html> (Official Guide)

The above [Merge request](#) mentions all the tasks that are needed for migrating to GTK4. These tasks are in accordance with the official [GTK guide](#) for migrating from GTK3 to GTK4. Major tasks have been completed but some are still left that are needed to be completed. The tasks and their description are mentioned below.

1. **GdkPixbuf** is deemphasized

GdkPixbuf is mostly used for getting the icons from in the form of pixbuf and creating, and scaling the thumbnails. As per the guide the use of **GdkPixBuf** should be replaced with pre-existing **GIcon** in case of icons or could be converted to newly introduced **GdkTextures** using [gdk_texture_new_for_pixbuf\(\)](#), this return object of **GdkTextures** which can satisfy the current needs.

```

@property
def icon(self):
    pixdir = os.path.join(get_pixmap_dir(), "effects")
    try:
        # We can afford to scale the images here, the impact is negligible
        icon = GdkPixbuf.Pixbuf.new_from_file_at_size(
            os.path.join(pixdir, self.effect_name + ".png"),
            ICON_WIDTH, ICON_HEIGHT)
    except GLib.Error:
        icon = GdkPixbuf.Pixbuf.new_from_file_at_size(
            os.path.join(pixdir, "defaultthumbnail.svg"), ICON_WIDTH, ICON_HEIGHT)

    return icon

def __create_scaled_thumb(self):
    """Creates scaled thumbnail from the original thumbnail."""
    try:
        thumb = GdkPixbuf.Pixbuf.new_from_file(self.get_thumb_path(self.uri,
ORIGINAL_THUMB_DIR))
        scale_pixbuf(thumb, SCALED_THUMB_WIDTH, SCALED_THUMB_HEIGHT)
        thumb.savev(self.get_thumb_path(self.uri, SCALED_THUMB_DIR), "png", [], [])
    except GLib.Error as e:
        self.warning("Failed to create scaled project thumbnail: %s", e)

```

(Example of current major use of GdkPixbuf)

2. Use GtkFixed instead of GtkLayout

GtkLayout implements **GtkScrollable** and hence it was used for making Scrollable widgets but now the **GtkScrolledWindow** can deal with widgets that do not implement **GtkScrollable** so **GtkFixed** could easily replace **GtkLayout** for placement of child widgets.

3. Viewer (Reliant on GtkSink port)

GtkSink is implemented in GTK4 which can be found [here](#). Required changes need to be made in sink widgets in [pitivi/viewer](#).

4. Symbolic icons colour change issue

In GTK4 the icons that are directly stored in `$APP_ID/icons/` will be treated as unthemed icons and will not be recoloured resulting in colour change issues hence the location of the icons need to be changed so that they get recoloured and icon resources can have icon theme semantics.

5. GtkPopover

Some of the popover children are attached to the parent widgets of type `GtkPopoverMenuBar` or `GtkMenuButton` while some are attached to make a custom widget that uses popover. As per the changes in `GtkPopover`, the parent widget that is of the supported type needs no changes while the custom widgets need to call `gtk_popover_present()` so that the parent widget remains aware of its popover children and manages their size allocation.

```
...
# Add effect popover button
self.effect_popover = EffectsPopover(app)
self.add_effect_button = Gtk.MenuButton(_("Add Effect"))
self.add_effect_button.set_popover(self.effect_popover)
self.add_effect_button.props.halign = Gtk.Align.CENTER

self.object_tracker_box = Gtk.ButtonBox()
self.object_tracker_box.props.halign = Gtk.Align.CENTER

self.cover_popover: Optional[Gtk.Popover] = None
self.cover_object_button: Optional[Gtk.MenuButton] = None
if "cvtracker" not in MISSING_SOFT_DEPS:
    self.cover_object_button = Gtk.MenuButton(_("Cover Object"))
    self.object_tracker_box.pack_start(self.cover_object_button, False, False,
0).
```

(example use case of popover attached to `Gtk.MenuButton`)

6. Drop the use of `Dialog.run` (Partially done)

Use of `dialog.run()` should be dropped rather than `dialog.show()` could be used by connecting to the `GtkDialog::response` signal. The example usage is implemented in the MR, some remaining minor changes should be done.



```

---      dialog.run( )
---      dialog.destroy( )
+++      dialog.connect( "response", _dialog_response_cb)
+++      dialog.show( )

```

(example replacement of `dialog.run()` from MR)

7. Implement Drag and Drop EventController

Both source and destination side Drag and Drop APIs have been changed and event controllers should be used instead. The proper usage is given in the [guide](#).

8. Drop use of GdkScreen

The `GdkScreen` object has been removed in GTK4. Most APIs already have a replacement in GTK3 and some that don't have a replacement have been added to `GdkDisplay`. Both have a method name `get_default()` which does the same job and is an easy replacement. Ref: [Gdk.Screen.get_default\(\)](#), [Gdk.Display.get_default\(\)](#)



```

def __setup_css(self):
    css_provider = Gtk.CssProvider()
    css_provider.load_from_data(EDITOR_PERSPECTIVE_CSS.encode("UTF-8"))
---    screen = Gdk.Screen.get_default()
+++    screen = Gdk.Display.get_default()
    style_context = self.app.gui.get_style_context()
    style_context.add_provider_for_screen(screen, css_provider,
                                         Gtk.STYLE_PROVIDER_PRIORITY_APPLICATION)

```

(Required Changes) (only use of `Gdk.Screen` in for `get_default()`)

9. Update to GtkFileChooser API changes

As per the guide `GtkFileChooser` has been moved to `GFile-based API`, due to which APIs for getting path name or URI have also been changed and

`GtkFileChooser` does not support custom widget anymore. There are also some more changes that are not applicable to Pitivi's UI as those APIs are not used. The major use is of `file_chooser` API for providing the URI.

```
def update_preview_cb(self, file_chooser):
    """Previews the URI of the specified file chooser.

    Args:
        file_chooser (Gtk.FileChooser): The file chooser providing the URI.
    """
    # TODO : Change API as per Guide
    uri = file_chooser.get_preview_uri()
    previewable = uri and uri_is_valid(uri)
    if not previewable:
        self.clear_preview()
        return
    self.preview_uri(uri)
```

(part of code that needs to be changed)

10. Adapt to GtkHeaderBar API changes

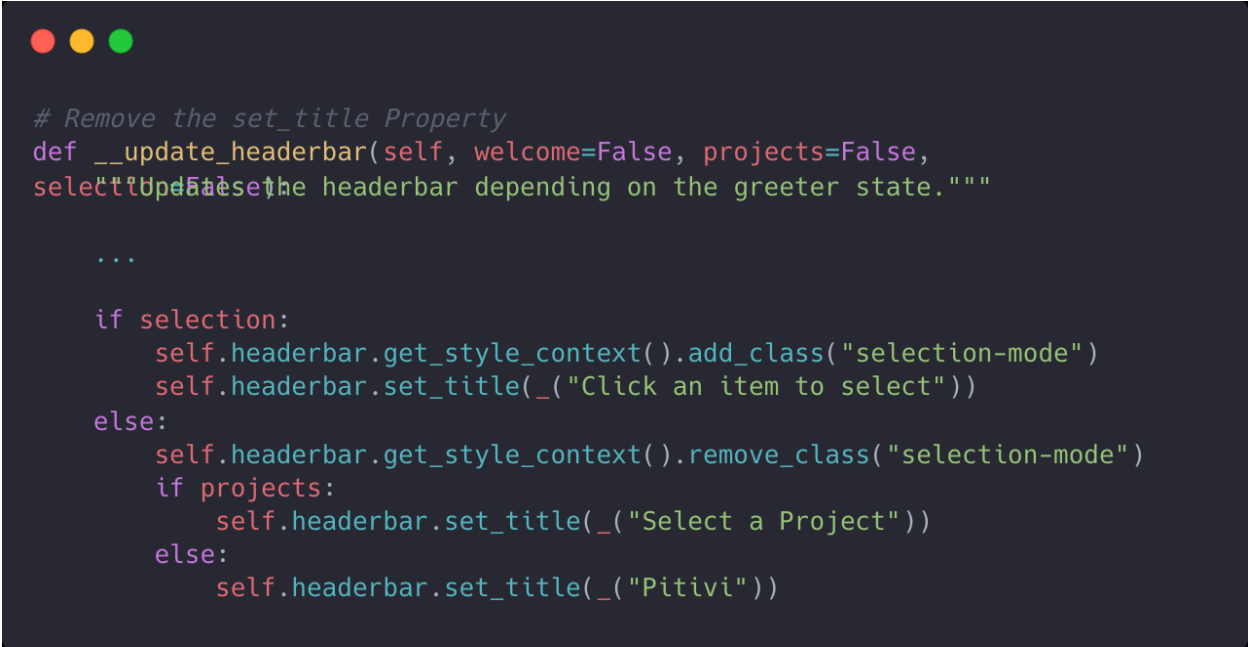
Major renames and changes have been done in the `GtkHeaderBar` API. For Pitivi's codebase, the major changes include rename of `gtk_header_bar_set_show_close_button()`.

The `gtk_header_bar_set_title()` method has been removed along with the getter and setter so renaming of the window should be done using the `GtkHeaderBar:title-widgit` property to add `GtkLabel` as shown [here](#).

```
# Rename the Function
def __create_headerbar(self):
    headerbar = Gtk.HeaderBar()
    headerbar.set_show_close_button(True)

    undo_button =
    Gtk.Button.new_from_icon_name(
```

(part of source-code where `set_show_close_button()` should be renamed)



```
# Remove the set_title Property
def __update_headerbar(self, welcome=False, projects=False,
                        selection=False):
    """Updates the headerbar depending on the greeter state."""

    ...

    if selection:
        self.headerbar.get_style_context().add_class("selection-mode")
        self.headerbar.set_title(_("Click an item to select"))
    else:
        self.headerbar.get_style_context().remove_class("selection-mode")
        if projects:
            self.headerbar.set_title(_("Select a Project"))
        else:
            self.headerbar.set_title(_("Pitivi"))
```

(example: part of source code where `set_title` is used and needs to be changed as mentioned above)

11. Fix Interactive Intro (Reliant on GtkPopover changes)

The Interactive Intro feature of Pitivi extensively uses popover widgets to give descriptions of different components and the use of the software. As [GtkPopover](#) is changed as mentioned in point 5 required changes should be made in the [pitivi/interactiveintro.py](#) method as per the.

12. Replace "configure-event" and "size-allocate" signals

Event controllers and gestures replace event signals in GTK4. In our code base, **configure-event** is used which should be replaced and **size-allocate** is also removed which should be replaced with [GtkDrawingArea::resize](#) or [GtkGLArea::resize](#).

Updating the Unit Tests

Some unit tests are aware of objects being GTK3 widgets, checking properties, mocking fields and calling methods to simulate GTK3 events. These shall be updated accordingly so they continue to do their job.

Bug Fixing during migration

During the migration of Pitivi to GTK4, it is possible that there will be numerous critical and non-critical bugs that will need to be addressed for a successful migration. These bugs can only be identified once the migration is completed and thorough testing is performed. It is imperative to identify and fix these bugs to ensure the smooth operation of the new version. The number and nature of the bugs will depend on the specific feature and code changes made during the migration, and the severity of the bugs will determine their priority for fixing. A comprehensive testing strategy will be necessary to address these bugs efficiently.

Another important task during the migration would be to run Pitivi and fix all the non-critical GTK warnings that appear to polish the codebase. This step will help ensure the code is up to date with the latest GTK API, and that the codebase is more efficient.

Stretch goals

As mentioned on this [page](#) Pitivi's dependencies are **cross-platform**, so in theory, it should be easy to bundle the app and the libs and ship it.

My secondary goal after the completion of the primary goal would be to port Pitivi on **Mac OS** using **GStreamer's Cerbero** build system for a building. I have already contributed to Cerbero for this idea to become successful. It could be achieved by merging MR [#113](#) of [Cerbero](#) for which MR [#640](#), [#639](#) and [#638](#) should get merged. Luckily all these MRs are merged and after the successful merging of [#113](#) it could be possible to write a recipe for building GTK4 and finally write the recipe for building Pitivi on Mac Os as mentioned in Issue [#2389](#).

As I have already worked a little bit on this idea and have tried to contribute to Cerbero after the completion of my primary goals I would like to work on this idea of porting Pitivi to Mac OS and helping it get one step closer to truly becoming a cross-platform video editor.

What are your past experiences with the open-source world as a user and as a contributor?

As a user, I have had many experiences with open-source software. The first open-source software that I used as a kid was the VLC player. Amazed by the quality of the software, the features it provides and the fact that it is free for all to use and contribute I started using other open-source software and Operating Systems (currently Ubuntu 22).

I have been contributing to open source projects for the last 1 year after I learnt programming in my 1st University course and I have learned a lot by working on these projects as they provide the support of experienced developers and experience to work on software used in real life and to contribute to them.

- **Pitivi** : To explore the various components of the **Pitivi** project and get familiar with the codebase, I made some contributions to the project.

MRs:

https://gitlab.gnome.org/GNOME/pitivi/-/merge_requests/464 (Merged)

https://gitlab.gnome.org/GNOME/pitivi/-/merge_requests/465 (Draft)

I have started working on my GSOC project. I am currently working on the “**Replace GtkLayout replacement required**” task mentioned in the following MR [#442](#).

- **Cerbero** : I also contributed to the MR [#639](#) of [Gstreamer/Cerbero](#) which also helped in the merge of [#638](#).

https://gitlab.freedesktop.org/gstreamer/cerbero/-/merge_requests/639



I have contributed to the following open-source software so far.

- **Godot Engine:** I worked on Godot which is an open-source, Multi-Platform 2D and 3D game engine, written in C++ and uses OpenGL3.
<https://github.com/godotengine/godot/pull/70396> (merged)
<https://github.com/godotengine/godot/pull/71382> (merged)
<https://github.com/godotengine/godot/pull/71910> (merged)
<https://github.com/godotengine/godot/pull/72329> (accepted but later superseded)
- **Rootex:** I worked on Rootex which is an open source 3D game engine written in C++ and uses DirectX 11. (Explained in detail [here](#))
<https://github.com/sdslabs/Rootex/pull/579> (merged)
<https://github.com/sdslabs/Rootex/pull/603> (merged)
- **Quizio:** Quizio is an online platform for conducting exams and to conduct recruitments for students of IIT Roorkee.
<https://github.com/sdslabs/Quizio-Backend/pull/42> (merged)
<https://github.com/sdslabs/Quizio-Frontend/pull/185> (merged)

What other relevant projects have you worked on previously and what knowledge have you gained from working on them?

I have also worked on various other personal projects and learnt many new concepts of Computer Science and new tech stacks through them.

Rootex

- Rootex is an advanced C++ 3D game engine powering an in-production game yet to be announced. The game will finally ship on Windows and use DirectX 11. The engine is based on the Entity-Component-System architecture and uses a Lua scripting Engine for making games.
[GitHub](#) | [Docs](#)
- The engine was also covered by GamesFromScratch ([Link](#))
- My contribution to Rootex covers optimizing the shaders, implementing Tangent Space Normal Mapping in the shaders, and bug fixes. For this task, I learnt basic Computer Graphics and DirectX 11, HLSL and somewhat OpenGL. Also, this was the 1st time where I worked on software completely built using C++.

PaperTrading Platform

- PaperTrading Platform acts as a playground for newcomers to the world of the stock market. It helps them carry out virtual trade of stocks with real-time data and basic parameters for performance analysis and without risk of loss of real money.
- Frontend is written in React Js and the backend in Python's Django Framework.

[Github](#)

- This gave me a chance to learn Django and ReactJs along with that I learnt the working of Django-REST API and REST-API in general.

Quizio

- Quizio is an online platform for conducting exams and to conduct recruitments for students of IIT Roorkee. Frontend of the app is implemented in React Js and the backend is in Node.js powered by the MongoDB database.
- Worked in Quiz creation flow in frontend and writing APIs for Quiz, Section, Question creation and publishing, bug fixing and testing.
- Got experience working on a large-scale CRUD app in use.

ChainPe

- ChainPe is a Non-Custodial wallet which lets you pay to any UPI ID via your cryptocurrency balance, which makes it possible to buy anything and pay any amount to any UPI ID. Where the receiver receives fiat payment and the spender spends crypto. The payment speed is the same as any normal UPI transaction
- This was the entry point in my life after which I got to learn and build on blockchain in various other events and projects.

This project was also a **winner** at the **ETHIndia'2022** hackathon.

[Project Link](#)

Pipelined Processor and 3D Cache Simulator

- Simulating a 64-bit, 5 Staged Microprocessor along with making our own ISA (Instruction Set Architecture) for optimal performance while

doing basic mathematical operations like calculating the factorial of a number as large as 20! Which is not possible on normal calculators. Written in Verilog along with our assembler in C++.

[Github](#)

- Simulating an NxM (N-ways and M-set) Set-Associative Cache for benchmarking and calculating the hit rate and miss rate of any memory trace provided. Also written in Verilog.

[Github](#)

- These projects were part of my CSN-221 (Computer Architecture and Microprocessors). It had a very steep learning curve and at last, I ended up learning a lot about computer architecture and Hardware Description Language like Verilog.

Other Projects

- **ImageEditor:** Written in Python using PyQt5 framework. It can edit images and convert from one type to another (Eg: jpg to png). | [Github](#)
Learned about the Qt framework for developing frontend desktop apps using Python along with how images are stored and edited in computers, what is the difference and importance of different image formats like jpeg, jpg, png etc and their manipulation.
- **RsPainter:** Model rendering Graphics pipeline written in C++ using OpenGL. | [Github](#)
- Learned in depth about [OpenGL](#) and core concepts of Computer Graphics along with developing graphics basic graphics pipeline in C++.

Other Achievements

- **Inter-IIT Tech Meet 2023 Gold:** During the Inter-IIT Tech Meet held in January-February 2023, I had the opportunity to work on a problem statement proposed by **ConsenSys**. Our team focused on developing a suite of MetaMask snaps that provided enhanced functionality, including smart account, two-factor authentication, transaction simulation, multi-signature capabilities and many more.
- **ETHIndia 2022:** I participated in one of Asia's biggest Ethereum hackathons, ETHIndia in 2022 where we were the top 12 world finalists and the **winners** of the ETHIndia'22 Hackathon. We built a non-Custodial wallet using which you can pay to any UPI ID from your cryptocurrency balance.

How do you plan to schedule the work on the project?

As a college student, I intend to utilize my summer breaks by working on a project. After completing my university exams at the end of April, I plan to start working on the project in the first week of May. I have scheduled to work for 4-5 hours a day between 4 pm IST (UTC+5.30) to 10 pm IST (UTC+5.30) for 6 days a week, with Sunday being my day off. This schedule will allow me to balance my project work with other responsibilities while ensuring steady progress towards achieving project goals.

Following is the project timeline for my GsoC. I would be taking up the 175-hour format.

Pre GSOC Period	
4th April - 4th May	● I would be contributing to Pitivi by solving issues and completing my remaining MR #465 to get more familiar with the

	codebase. I will start working on MR #442.
Community Bonding Period	
4th May - 28th May	- Get to know more about the Pitivi community - Discussion with my mentor to refine the plan of migration of the whole UI. - Identify any possible bugs or changes that might be required in the code-base other than what is mentioned in the guide.
Coding period	
29th May - 2nd June	Completing subtasks 1 and 2 for migration
3rd June - 6th June	Completing subtasks 3 and 4 for migration
7th June	Buffer time for completion of remaining tasks (if any)
8th June - 11th June	Completing subtasks 5 and 6 for migration
12th June - 15th June	Completing subtasks 7 and 8 for migration
16th June	Buffer time for completion of remaining tasks (if any)
17th June - 20th June	Completing subtasks 9 and 10 for migration
21st June - 24th June	Completing subtasks 11 and 12 for migration
25th June	Buffer time for completion of remaining tasks (if any)
26th June - 2nd July	Fixing the unit tests for GTK4 port
3rd July - 4th July	Buffer time for completion of remaining tasks (if any)
5th July - 7th July	Getting review from a mentor for the work till now and preparing for migrating Pitivi to GTK4



8th July - 10th July	Successfully migrating Pitivi to GTK4 if everything goes well or making changes as suggested by a mentor.
10th July - 14th July	Midterm evaluation period
15th July - 17th July	Completing migration of Pitivi to GTK4
18th July - 20th August	Testing the UI for any possible bugs. Bug reporting and bug fixing Start working on secondary goals if successfully completing the Primary goal

I have carefully allocated enough time to each task, including my secondary goal, and have factored in ample time for testing and bug fixing from July 18th to August 20th. Additionally, I have kept some buffer days between tasks to account for any unforeseen delays or if any of the previous tasks require additional time. This will ensure that even if any critical bugs arise, I can handle them without impacting the project's overall progress.